
Near-Failure Latent Corrections as a Training Curriculum for Deep RL

Shou-Jen Chen
Department of EECS
University of California, Berkeley
lawrancechen@berkeley.edu

Extended Abstract

Robot manipulation tasks contain bottleneck sub-tasks such as grasping and contact establishment where random exploration rarely succeeds. Standard off-policy RL fills the replay buffer with low-reward transitions at these states, and the critic learns poor Q-estimates where good estimates matter most.

We train a small latent correction module concurrently with an SAC base actor from scratch. A near-failure heuristic trigger detects stalls via a sliding-window range check on geometric signals in the observation (hand-to-object distance, object-to-goal distance). When the trigger fires, the correction substitutes an action from a policy conditioned on $\mathbf{z}' = \mathbf{z} + \Delta\mathbf{z}$ during rollout. The transition enters the replay buffer as a normal step and, if it reflects better task progress, produces stronger Bellman targets at stall states that feed into the SAC critic update. As the policy improves it stalls less, so the trigger fires less: the trigger rate drops from $\sim 79\%$ of steps early in training to below 15% at convergence, with trigger parameters fixed throughout.

Co-adaptation. Naive concurrent training causes co-adaptation: the actor encoder learns latents suited to the correction module rather than independently good actions. On pick-place-v3 seed 42, this causes the base actor to drop from 96% to 36% success when the correction is removed, falling below a plain SAC baseline without any correction.

Gradient isolation. A stop-gradient on the actor encoder during the correction update fixes this. The update uses three separate steps per iteration: (1) standard SAC critic, (2) standard SAC actor with no correction in the computation graph, (3) correction loss with the encoder output detached. Without gradient isolation, concurrent correction fails on all seeds (0/4); with it, all four seeds converge on pick-place-v3. At convergence, the base actor performs equally with and without the correction active (pick-place mean: 95% base-only vs 93.5% triggered), showing the benefit was absorbed into the base policy through the critic.

Results. On pick-place-v3 and push-v3 in Meta-World 3.0 (four seeds each, 2.5M steps), mean final SR rises from 66.0% to 93.5% on pick-place and from 32.5% to 89.5% on push. Gains come from seeds where baseline SAC fails; seeds where baseline SAC already converges regress by 2–4 pp.

Ablations. A random trigger at 5% fire rate produces results that matches the baseline (36% vs 32.5% on push-v3); near-failure targeting is the essential ingredient. Sequential correction (load converged baseline, freeze actor, train correction for 1.5M steps) reaches 65.0% mean final SR on pick-place, below the 66.0% frozen baseline, and rescues no failed seed, at 60% more compute. SAC+HER reaches 9.5% mean final SR on push-v3; the 80 pp gap likely reflects a dense–binary reward mismatch that HER was not designed for. Action-space residual correction collapses the entropy temperature before the bottleneck is discovered; latent correction avoids this because $\Delta\mathbf{z}$ acts before action sampling.

Limitations. On assembly-v3, the trigger fires at 26–74% of steps across seeds, collapsing entropy before task discovery; our method fails on all four seeds. Each task requires a hand-designed trigger

subclass; removing this requirement was a goal. Two approaches were tried and both failed. A Q-advantage-based learned trigger produced always-positive training labels since $\Delta Q > 0$ for nearly all states at convergence. A trigger based on $\|\Delta \mathbf{z}\|$ magnitude failed from bootstrapping circularity: the trigger fires when $\|\Delta \mathbf{z}\|$ is large, but $\Delta \mathbf{z}$ stays small until the trigger fires, so neither starts. Code: <https://github.com/lawraa/latent-recovery-rl>.

1 Introduction

Robot manipulation tasks typically decompose into sequential sub-tasks, each of which must be solved for the episode to succeed. Grasping in pick-and-place and establishing contact in pushing are examples of bottleneck sub-tasks: they are prerequisite for all subsequent progress, yet random exploration produces successful transitions for them only rarely. A policy trained with standard off-policy RL on these tasks spends much of its early training generating low-reward experience at bottleneck states where the base policy stalls without progress. The replay buffer accumulates mostly uninformative transitions at these states, and the critic learns weak Q-value estimates precisely where better estimates would be most useful.

We study the setting where a lightweight task-progress signal is available (for example, hand-to-object distance or object-to-goal distance) and ask how to use it to improve off-policy RL training specifically at bottleneck states. One way to address this is to produce better-quality transitions at these hard states. Hindsight Experience Replay [Andrychowicz et al., 2017] does this by relabeling failed trajectories with achieved states as substitute goals, providing additional reward signal. We take a different approach: a small latent correction module, trained concurrently with the base actor from scratch, substitutes a sample from a policy conditioned on a corrected latent whenever a near-failure heuristic detects that the policy is stalling. The resulting transition enters the replay buffer as a normal rollout step; if it reflects better task progress, it produces stronger Bellman targets at stall states, which in turn benefits the base actor via the standard SAC actor update. The intervention is self-regulating: the trigger fires heavily in early training when the policy stalls frequently on bottlenecks, and its fire rate declines naturally as the policy becomes more competent, with trigger parameters held fixed throughout.

The critical engineering challenge in this setup is that the correction is trained *concurrently* with the base actor, not on top of a frozen pretrained policy. The most closely related residual and corrective policy methods keep the base frozen by design: Silver et al. [2018], Policy Decorator [Yuan et al., 2025], and RMA [Kumar et al., 2021] all prevent gradient interaction between the correction and the base. Concurrent training is desirable here because the base actor benefits from the correction’s improved transitions during its own learning; a sequential protocol where the base is trained to convergence before the correction is introduced would not provide this benefit during the critical early training period (see Appendix E for empirical verification). But naive concurrent training causes *co-adaptation*: the actor encoder receives gradient signal from both the SAC actor loss and the correction loss, and begins learning correction-compatible representations rather than independently good actions. When the correction is removed, the base actor collapses from 96% to 36% success – below the performance of a plain SAC baseline that never had a correction module at all.

The fix is a single architectural choice: a stop-gradient on the actor encoder during the correction update step. This severs the correction loss’s gradient path to the encoder, keeping the base actor exclusively trained by the standard SAC actor loss. The primary effect is *training reliability*: across four random seeds, decoupled concurrent training succeeds on every seed while the naive jointly-trained version fails on all four. The corrected transitions continue to produce stronger Bellman targets throughout training. A useful consequence of this design is that the base actor at convergence is also independently capable: removing the correction at evaluation incurs no performance drop, confirming the benefit of corrected transitions was absorbed into the base policy rather than producing encoder representations that depend on the correction module.

Contributions. (1) We propose concurrent decoupled training of a near-failure latent correction module: at stall states the correction substitutes actions to produce stronger Bellman targets, while a stop-gradient on the actor encoder keeps the base actor independently trainable. (2) We identify and quantify co-adaptation in naive concurrent training: without gradient isolation, the base actor collapses from 96% to 36% success when the correction is removed. (3) We show gradient isolation is primarily a *training stability* requirement: it enables reliable multi-seed convergence on both

tasks, and as a consequence the base actor at convergence performs equivalently with and without the correction at deployment. (4) On pick-place-v3 and push-v3 (four seeds each), our method raises mean final success rate on both tasks primarily by rescuing seeds where baseline SAC fails; a comparison against Hindsight Experience Replay shows that generic replay relabeling does not address the same training bottleneck.

2 Related Work

Curriculum learning in RL. Curriculum learning in RL typically operates by changing the task distribution presented to the agent: ordering tasks by difficulty [Bengio et al., 2009], selecting tasks proportional to learning progress [Graves et al., 2017], or adaptively generating environment parameters at the frontier of the agent’s competence [Portelas et al., 2020]. A survey of these methods [Narvekar et al., 2020] identifies task ordering, starting-state selection, and goal generation as the dominant mechanisms – but none operates by injecting modified transitions into the experience replay buffer during off-policy training. Our approach complements these methods: rather than choosing what task to present, we augment which transitions enter the replay buffer during a fixed task, specifically targeting the bottleneck states where the policy is near failure.

Replay buffer augmentation. Hindsight Experience Replay [Andrychowicz et al., 2017] augments the replay buffer by relabeling failed trajectories with the states the agent actually reached as substitute goals, providing additional successful sparse-reward training signal in goal-conditioned settings. RLPD [Ball et al., 2023] seeds the online replay buffer with offline transitions to reduce the data needed for online learning. Our method improves replay buffer quality through a different mechanism: at stall states identified by a task-progress heuristic, the correction replaces the base policy’s action with a corrected action during rollout, producing transitions with better rewards and next states rather than relabeling goals or drawing from a fixed offline dataset. We compare directly against HER in Section 5.5; in the dense-reward Meta-World setting HER underperforms even plain SAC, consistent with the dense–binary reward mismatch discussed there, and the 80 pp gap to our method reflects different mechanisms rather than a matched comparison.

Residual and corrective policies. Silver et al. [2018], Johannink et al. [2019] add a learned correction in action space on top of a fixed base policy. Policy Decorator [Yuan et al., 2025] adds an action-space residual on top of a frozen pretrained imitation learning model and fine-tunes it online. Rana et al. [2023] learn residual skill policies in a demonstration-derived latent skill space, adapting pre-learned skill embeddings rather than raw actions. Dodeja et al. [2026] extend residual RL with an uncertainty-based gate: the residual is applied only when the current state is far from the demonstration dataset or has high ensemble variance across base policies, with the threshold decaying exponentially during training. Each of these works keeps the base policy or controller frozen, so there is no gradient interaction between the correction and the base, and no curriculum effect during base training. Our method differs from all of them in training the correction *concurrently* with the base from scratch, which is what enables the training benefit. Our trigger mechanism also differs from uncertainty-based gating: rather than measuring distance to a demonstration dataset, we detect task-progress stalls directly from geometric signals in the observation, and the trigger rate declines naturally as the base policy improves rather than on a fixed exponential schedule.

Training-only auxiliary policies. Leave No Trace [Eysenbach et al., 2018] trains a reset policy concurrently with the forward policy: the reset policy improves training by returning the agent to safe starting states, but is discarded at deployment. Jump-Start RL [Uchendu et al., 2023] uses a guide policy to handle the early steps of each episode, steering the agent into informative states that enter the replay buffer, then withdraws as the main policy matures. Both share the core structure of our approach – an auxiliary module active during training that improves the data distribution seen by the main policy and is not needed at deployment. The key difference is that both prior works use completely separate networks; gradient isolation is achieved trivially because there are no shared parameters. Our correction shares the actor’s heads and critic at update time, so gradient isolation requires an explicit stop-gradient on the encoder.

Intervention and recovery policies. Recovery RL [Thananjeyan et al., 2021] trains a recovery policy that overrides task actions when the agent enters an estimated unsafe region; both the task and

recovery policies are retained at deployment. SAILR [Wagener et al., 2021] uses a Q-advantage-based criterion to decide when a guide policy takes over and specifically studies whether the task policy can be deployed without the intervention after training, which is the same base-actor independence question we investigate empirically. Recovery RL retains both policies at deployment; SAILR allows the intervention to be removed, making it structurally closer to our training-curriculum framing. Our method shares the intervention structure (a trigger-based module overrides base-policy actions at selected states), but differs in purpose and design: the trigger targets task-progress stalls rather than safety violations, uses a geometric heuristic rather than a learned Q-advantage criterion, and the correction is a pure training curriculum: gradient isolation ensures the base actor internalizes the improved transitions so the correction is not required at deployment, a property we verify empirically in Section 5.4.

Gradient interference in concurrent auxiliary training. When two modules share parameters, conflicting gradient directions can corrupt each module’s representations [Lyle et al., 2021]. Lin et al. [2019] show that concurrent auxiliary task gradients can harm the primary policy and propose adaptive gradient reweighting as a remedy. Our approach uses a simpler and harder solution: a stop-gradient on the shared encoder, which eliminates the interference entirely rather than damping it. This is necessary here because the correction and base actor have fundamentally different objectives and cannot be safely mixed through any weighting scheme.

Decoupled sequential training. RMA [Kumar et al., 2021] trains legged locomotion in two sequential phases: the base policy first with privileged encoder access, then an adaptation module via supervised regression from proprioceptive history with the base frozen. The explicit motivation for sequential training is to prevent the adaptation module from corrupting base representations – the same problem we solve with gradient isolation. Our approach differs in two ways: training is concurrent (not sequential), and the correction influences the replay buffer during base training rather than only after the base has converged.

Benchmark. We evaluate on Meta-World 3.0 [Yu et al., 2020] across four tasks: pick-place-v3 and push-v3 as primary evaluation tasks (four seeds each, full ablations), door-open-v3 as an easy task where both methods saturate, and assembly-v3 as a negative stress test for trigger saturation. Each task uses 50 MT1 training goal positions; evaluation sweeps all 50 deterministically (`reset_to_task(i)`, fixed order), producing reproducible success-rate estimates comparable across seeds.

3 Method

3.1 SAC Baseline

We use Soft Actor-Critic (SAC) [Haarnoja et al., 2018] with automatic entropy tuning. The actor is a squashed-Gaussian policy with a deliberate two-stage architecture:

$$\mathbf{z} = \text{enc}(\mathbf{s}), \quad \mu = W_\mu \mathbf{z}, \quad \log \sigma = \text{clamp}(W_\sigma \mathbf{z}, -5, 2)$$

where the encoder maps observations $\mathbf{s} \in \mathbb{R}^{39}$ to a latent vector $\mathbf{z} \in \mathbb{R}^{256}$ via LayerNorm-Tanh followed by one ReLU layer. The split into encoder and linear heads exposes $\mathbf{z}$ as an injection point for the correction module without modifying any other actor code. The actor’s forward method accepts an optional `z_override` argument; all correction paths use this hook exclusively.

The critic is a standard twin Q-network [Fujimoto et al., 2018]. Entropy temperature α is tuned automatically to target $\mathcal{H}_{\text{target}} = -|\mathcal{A}| = -4$. We include an $\alpha_{\min} = 0.05$ floor on the log-space entropy parameter to prevent premature entropy collapse on seeds where the initial network weights produce overly concentrated policies.

3.2 Latent Correction Module and the Curriculum Mechanism

The correction module $f_\phi : \mathbb{R}^{39+256} \rightarrow \mathbb{R}^{256}$ is a two-layer ReLU MLP that maps the concatenated observation and base latent to an additive correction:

$$\Delta \mathbf{z} = f_\phi([\mathbf{s}; \mathbf{z}]), \quad \mathbf{z}' = \mathbf{z} + \Delta \mathbf{z}$$

The output layer is initialized with orthogonal weights at gain 0.01 and scaled by 0.1, ensuring $\Delta \mathbf{z} \approx \mathbf{0}$ at training onset. An L2 regularization term $\lambda \|\Delta \mathbf{z}\|^2$ with $\lambda = 10^{-3}$ prevents unbounded growth.

How the correction improves base-actor training. When the trigger fires, the correction replaces the base policy’s action with $\mathbf{a}'$ sampled from the corrected policy $\pi(\cdot | \mathbf{s}, z_{\text{override}} = \mathbf{z}')$. This action is executed in the environment, and the resulting transition $(\mathbf{s}, \mathbf{a}', r', \mathbf{s}')$ enters the replay buffer as a normal rollout step. If $\mathbf{a}'$ makes better task progress than the base action would have (e.g., moves the hand closer to the object rather than stalling), then r' is higher or $\mathbf{s}'$ is a better state with higher future value, giving a stronger Bellman target when this transition is sampled during critic training. The base actor is then updated by Step 2 (standard SAC) using a critic that has learned higher Q-values at bottleneck states. The base actor therefore benefits from the correction’s improved transitions indirectly, through the critic, without directly receiving any gradient from the correction module. Concurrent training is desirable here because this benefit requires the correction to be active during the base policy’s learning; it cannot be retroactively added after the base policy has already converged.

3.3 Challenge: Co-adaptation under Concurrent Training

The straightforward concurrent design computes a single actor-correction loss:

$$\mathcal{L}_{\text{joint}} = \mathbb{E} \left[\alpha \log \pi(\mathbf{a}' | \mathbf{s}) - \min_i Q_i(\mathbf{s}, \mathbf{a}') \right] + \lambda \|\Delta \mathbf{z}\|^2$$

where $\mathbf{a}'$ is from the corrected policy, and both the actor and correction optimizers step on this loss. The problem is that $\nabla_{\text{enc}} \mathcal{L}_{\text{joint}}$ flows through f_ϕ : the encoder learns to produce latents that serve as good inputs for the correction module, rather than latents that lead to good actions independently. On pick-place-v3 (seed 42, 2.5M steps), this causes the jointly trained base actor to drop from 96% to 36% success when the correction is removed at evaluation time – a 60-percentage-point collapse that leaves the base actor performing worse than a plain SAC baseline with no correction.

3.4 Gradient Isolation Enables Concurrent Training

The fix is to block the correction loss’s gradient path to the actor encoder, using three separate update steps per iteration:

Step 1 (critic): Standard SAC Bellman target, base actor used for next-state actions.

Step 2 (base actor): $\mathcal{L}_{\text{actor}} = \mathbb{E} \left[\alpha \log \pi(\mathbf{a} | \mathbf{s}) - \min_i Q_i(\mathbf{s}, \mathbf{a}) \right], \quad \mathbf{a} \sim \pi_{\text{base}}(\cdot | \mathbf{s}).$

Step 3 (correction): $\mathbf{z}_{\text{base}} = \text{no_grad}(\text{enc}(\mathbf{s})),$

$$\mathcal{L}_{\text{corr}} = \mathbb{E} \left[\alpha \log \pi(\mathbf{a}' | \mathbf{s}) - \min_i Q_i(\mathbf{s}, \mathbf{a}') \right] + \lambda \|\Delta \mathbf{z}\|^2,$$

$$\mathbf{a}' \sim \pi(\cdot | \mathbf{s}, z_{\text{override}} = \mathbf{z}_{\text{base}} + \Delta \mathbf{z}).$$

The `no_grad()` wrapper in Step 3 creates $\mathbf{z}_{\text{base}}$ with no gradient function, so $\partial \mathcal{L}_{\text{corr}} / \partial \text{enc} = 0$. The actor encoder is updated only by Step 2 (standard SAC, no correction in graph). The correction is updated only by Step 3 (frozen encoder, gradients reach only f_ϕ). Table 1 summarizes the gradient isolation.

With this design, the corrected actions still enter the replay buffer and produce stronger Bellman targets at bottleneck states, benefiting the base actor indirectly through the critic. At the same time, the encoder is protected from correction-gradient contamination. The base actor is independently capable by construction.

Table 1: Gradient flow per update step. A checkmark indicates autograd gradients flow through the module during the backward pass. Only the step’s own optimizer updates parameters: `actor_optim` in Step 2; `correction_optim` in Step 3. Actor heads appear in the Step 3 autograd graph (via z') but are not parameter-updated by `correction_optim`.

Update step	Actor encoder	Actor heads	Correction f_ϕ
Step 2 (actor)	✓	✓	
Step 3 (correction)		✓	✓

3.5 Near-Failure Heuristic Trigger

A lightweight, task-aware trigger decides when to apply the correction during rollouts. The trigger has no learned parameters; it detects progress stalls from geometric signals in the observation and environment info dictionary.

For pick-place-v3, the task has two structural phases. In Phase A (pre-grasp, `grasp_success = 0`), the trigger tracks $d_{\text{hand}} = \|s_{0:3} - s_{4:7}\|_2$ (hand-to-object distance). In Phase B (post-grasp, `grasp_success = 1`), it tracks $d_{\text{goal}} = \text{info}[\text{'obj_to_target'}]$. A drop event (grasp flipping $1 \rightarrow 0$ while $d_{\text{goal}} > 0.05$ m) fires immediately.

The stall criterion uses a sliding window of length $W = 20$ steps: the policy is stalled if $\max(\text{history}) - \min(\text{history}) < \delta_{\min} = 0.01$ m. Using the range rather than the first-vs-last difference makes the detector robust to oscillatory motion that never makes net progress. Once any condition fires, the trigger remains active for a cooldown of 15 steps. All trigger logic is suppressed within 0.05 m of the goal.

For push-v3, the trigger has two phases: approach stall (hand-to-puck distance, Phase A) and transport stall (puck-to-goal distance, Phase B, active when hand-to-puck ≤ 0.06 m). For door-open-v3, Phase B uses `info[\text{'in\_place\_reward'}]` because `obj\_to\_target` is hardcoded to zero in Meta-World 3.0’s door-open implementation. Each new task requires a subclass with task-specific phase logic, registered in a shared factory.

The trigger controls only *data collection*: the correction is applied during rollouts only when the trigger fires. Gradient updates (Steps 2 and 3) occur on every training step regardless of trigger state, ensuring the correction module receives dense gradient signal throughout training.

4 Experimental Setup

Environment. We evaluate on four tasks from Meta-World 3.0 [Yu et al., 2020]: **pick-place-v3** (grasp an object from a randomized position and place it at a randomized goal), **push-v3** (push an object to a randomized goal without grasping), **door-open-v3** (pull a door handle to open it), and **assembly-v3** (peg insertion with precise alignment) as a negative stress test to probe method limits under trigger saturation. We use the MT1 single-task protocol: each task exposes 50 goal positions via `MT1(task_name).train_tasks`, and we train and evaluate on all 50 of these positions (following the standard MT1 convention of training and evaluating on the same goal set). Observation dimension is 39; action dimension is 4 (end-effector Δxyz plus gripper). Episodes are truncated at 500 steps.

Training protocol. All runs train for 2.5M environment steps (1M for door-open, which saturates early) with a replay buffer of capacity 1M transitions, batch size 128, and one gradient update per step. The SAC hyperparameters follow the Meta-World community reference [Yu et al., 2020]: learning rate 3×10^{-4} , discount $\gamma = 0.99$, soft-update coefficient $\tau = 0.005$, and target entropy $\mathcal{H}_{\text{target}} = -4$. We set $\alpha_{\min} = 0.05$ and $\alpha_{\text{lr}} = 10^{-4}$ on all conditions to stabilize multi-seed training (see Section 6 for details). Correction module hidden width is 128; correction learning rate matches the actor (3×10^{-4}).

Evaluation protocol. Every 50,000 steps, we evaluate deterministically on all 50 task variants using `env.reset_to_task(i)` for $i = 0, \dots, 49$. Reported success rate is the fraction of the 50 episodes that achieved task success. This deterministic sweep produces a fully reproducible number comparable across seeds and conditions.

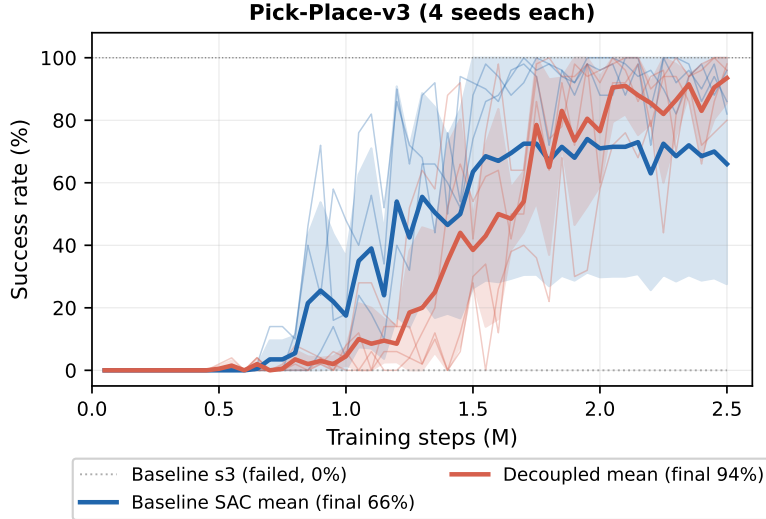


Figure 1: Pick-place-v3 learning curves (4 seeds each, 2.5M steps). Shaded bands: mean $\pm$ std over all 4 seeds for both conditions. Faint lines: individual seeds (s0–s2 in blue; s3 shown as dotted gray for baseline). Baseline seed 3 achieves 0% throughout 2.5M steps and pulls the baseline mean down; the same seed with decoupled training reaches 96% final success rate.

Conditions. For each task we run four seeds (0–3) of:

- **Baseline SAC:** plain SAC with no correction module.
- **Decoupled (Ours):** SAC with the decoupled latent correction and heuristic trigger.

For pick-place-v3 we also run (single seed 42) the **always-on correction** (joint training, correction applied at every step) as a diagnostic to measure co-adaptation. Push-v3 includes two ablations: a **random trigger** (correction applied at a fixed 5% rate per step, same decoupled training) to assess whether trigger specificity matters; and **SAC with HER** [Andrychowicz et al., 2017] (four seeds, her_ratio = 0.8, future strategy, binary relabeled reward) as a comparison to general replay augmentation.

Metrics. We report final success rate (SR) at 2.5M steps, peak SR, and steps to first reach 80% SR. All aggregate means include all four seeds, including seeds that failed to converge (e.g., baseline pick-place seed 3, which achieved 0% throughout 2.5M steps). For time-to-threshold statistics (steps to first reach 80% SR), we average only over seeds that reached the threshold and report the count of such seeds separately. We report trigger fire rate as *actual correction fraction* (heuristic trigger rate $\times$ 16, since each fire activates the correction for 16 consecutive steps due to the 15-step cooldown).

Infrastructure. Local runs use Apple MPS (M-series chip). Modal A100 runs are used for pick-place seeds 2 and 3 for time efficiency and to use up remaining credits; all other runs use MPS. All runs use the same codebase; device differences do not affect reported numbers.

5 Results

5.1 Seed Robustness: Pick-Place-v3

Figure 1 shows learning curves for four seeds of each condition. Table 2 summarizes aggregate statistics.

The central finding is that decoupled training succeeds on all four pick-place seeds (final SR $\geq$ 80%) while baseline SAC fails completely on one (seed 3, 0% throughout 2.5M steps). The same seed with decoupled training reaches 100% peak and 96% final SR. We attribute this rescue to the near-failure trigger firing heavily during the pre-grasp stall phase that seed 3’s initialization produces, generating

Table 2: Pick-place-v3 aggregate results (4 seeds each, 2.5M steps). Means include all 4 seeds for both conditions. “Seeds conv.” counts seeds with final SR $\geq 80\%$. Steps to 80% are averaged over seeds that reached 80%; s3 never reached 80% for baseline.

Condition	Seeds conv.	Mean final SR	Steps to 80% (mean)	Notes
Baseline SAC	3 / 4	66.0%	1,283k (s0–s2)	s3: 0% throughout
Decoupled (ours)	4 / 4	93.5%	1,775k (all)	All seeds converge

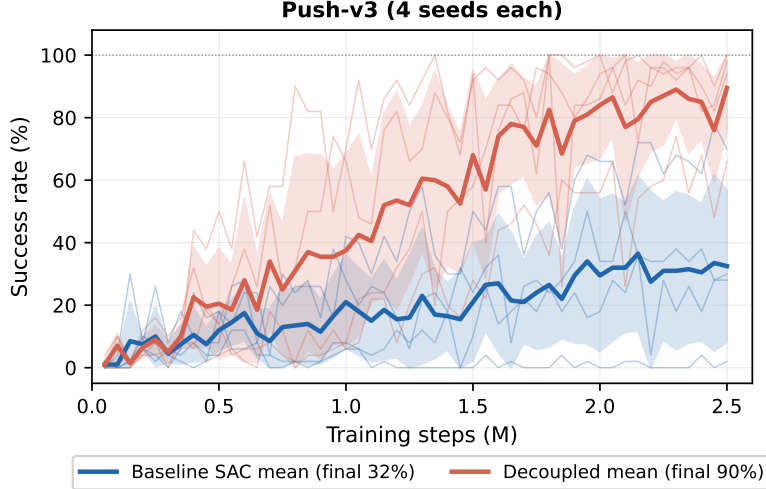


Figure 2: Push-v3 learning curves (4 seeds each, 2.5M steps). The gap between conditions is substantially larger than on pick-place: 57 percentage points in mean final success rate. Baseline seed 2 experienced a Q-value explosion (final 2%); decoupled seed 2 trains stably (final critic loss ≈ 600) and reaches 94%.

corrected approach-and-grasp transitions that improve the critic’s Q-estimates at stall states, a signal that baseline SAC rarely accumulates in early training on this seed.

On the three seeds where both conditions succeed (s0–s2), the baseline converges slightly faster to the 80% threshold (mean 1,283k vs 1,775k steps). For only converged seeds(s0–s2), mean final SR is 88.0% for the baseline and 92.7% for decoupled, so decoupled provides a modest improvement even among seeds that both methods solve. The overhead comes from the additional correction update step per iteration and the correction module’s mild anti-entropic effect on the actor. The decoupled advantage is primarily seed robustness, not speed.

5.2 Seed Robustness: Push-v3

Figure 2 shows push-v3 results using the two-phase trigger (Phase A: approach stall; Phase B: puck-transport stall when hand-to-puck ≤ 0.06 m).

Table 3: Push-v3 aggregate results (4 seeds, 2.5M steps). Decoupled uses the two-phase trigger (v2). “Seeds conv.” counts seeds with peak success rate $\geq 50\%$.

Condition	Seeds conv.	Mean final SR	Mean peak SR	Notes
Baseline SAC	2 / 4	32.5%	45.5%	s2: Q-explosion (2%); s1: 28%
Decoupled (ours)	4 / 4	89.5%	93.0%	All seeds converge

Push-v3 exposes the baseline more severely: only seeds 0 and 3 reach 50% peak success, while seed 2 suffers a Q-value explosion (critic loss $> 10^5$). Decoupled seed 2 trains stably and reaches 94%. An earlier single-phase trigger (approach only) failed to address the transport phase and performed similarly to the baseline; the Phase B transport trigger is required to rescue the full set of seeds.

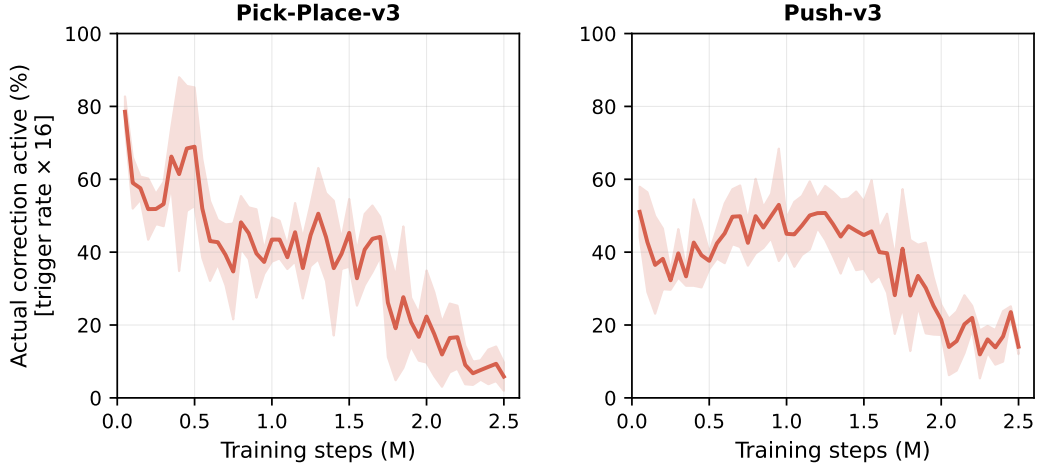


Figure 3: Actual correction fraction over training (mean $\pm$ std, 4 seeds each; trigger rate $\times$ 16 since each fire activates correction for 16 consecutive steps). **Left:** Pick-place-v3. The correction is heaviest early when the grasp bottleneck dominates, and self-regulates to $\sim 6\%$ by convergence. **Right:** Push-v3. The puck-transport Phase B trigger sustains a higher rate through mid-training before declining to $\sim 14\%$ by convergence. Trigger parameters are fixed throughout training in both tasks.

5.3 Trigger Rate as Curriculum Schedule

Figure 3 shows the actual correction fraction (mean $\pm$ std over 4 seeds) over training for both tasks. On pick-place-v3, the mean correction fraction starts at $\sim 79\%$ at step 50k, when seeds are stalling repeatedly on the pre-grasp bottleneck, and declines to $\sim 6\%$ by 2.5M steps as grasping becomes reliable. On push-v3, the initial rate is lower ($\sim 51\%$ at 50k), reflecting that push does not have an immediate grasp-bottleneck; the Phase B puck-transport trigger sustains a higher correction rate through mid-training ($\sim 44\%$ at 1M steps) before declining to $\sim 14\%$ by convergence.

This self-regulating profile is a consequence of the fixed trigger design: as the base policy improves, it stalls less and the trigger fires less, with no manual annealing of trigger parameters. By convergence, the correction fraction ranges from 3% to 12% across seeds on pick-place-v3 and 11% to 16% on push-v3 (Phase B fires longer during the puck-push contact phase).

5.4 Curriculum Internalization: Base-Actor Independence

We evaluate whether the correction’s benefit is absorbed into the base actor (training-time) or whether the base actor depends on the correction at deployment (co-adaptation). Table 4 and Figure 4 report base-off evaluation: the same trained checkpoint re-evaluated with the correction module disabled.

Table 4: Base-off evaluation at 2.5M steps (pick-place-v3, seed 42, diagnostic run). “Base only” re-evaluates the trained checkpoint with the correction module disabled. “Triggered” applies the correction only when the near-failure trigger fires. “Always-on” applies the correction at every step.

Condition	Deployment mode	Success rate
Baseline SAC	base only	80%
Always-on (joint)	corrected	96%
Always-on (joint)	base only	36%
Decoupled (ours)	base only	100%
Decoupled (ours)	triggered	100%
Decoupled (ours)	always-on	100%

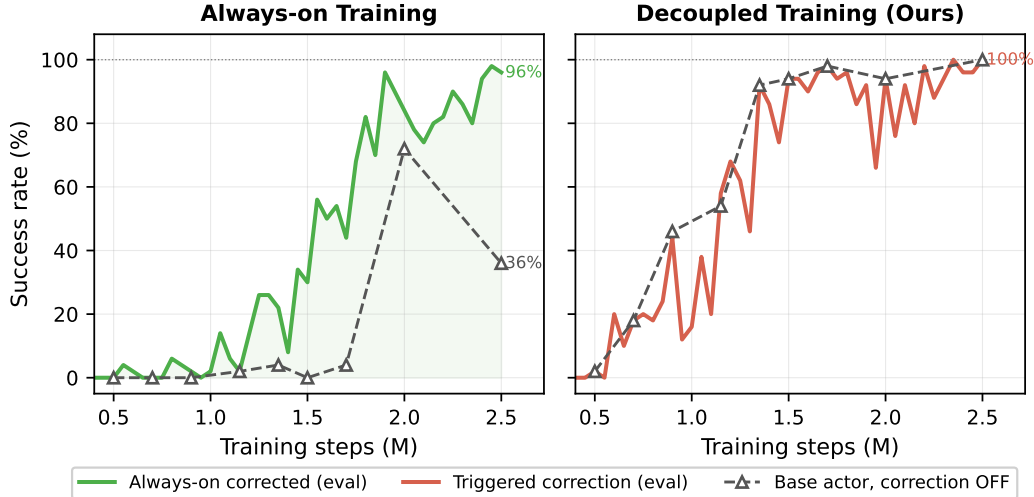


Figure 4: Diagnostic base-off comparison (pick-place-v3, seed 42). **Left:** Always-on jointly trained run. Solid: corrected eval; dashed with markers: base-off eval (correction disabled). The base actor collapses to 36% without correction – well below the 80% baseline – indicating the encoder has become dependent on the correction module. **Right:** Decoupled condition. Base-off and triggered evals overlap throughout, both reaching 100%.

Diagnostic: co-adaptation in a converged always-on run (seed 42). The multi-seed always-on runs (seeds 0–3) did not provide a suitable base-off co-adaptation test: three of four seeds ended at 0% success even with correction active, and one seed peaked at 78% before collapsing due to alpha instability. These runs demonstrate instability of coupled always-on correction, but not base-policy dependence. To diagnose whether co-adaptation can occur when always-on correction does converge, we examine an earlier run (seed 42) trained before the $\alpha_{\min} = 0.05$ stabilization. In this run, the corrected policy reached 96% success; disabling the correction at evaluation reduced success to 36%, indicating that the base actor had not independently internalized the behavior supported by the correction module.

Multi-seed confirmation: decoupled base actor is independently competent. Across all four pick-place-v3 seeds and all four push-v3 seeds, base-off SR equals or exceeds triggered SR at every seed (pick-place mean: 95%; push mean: 89.5%). The decoupled base actor matches full deployment performance without any correction module, confirming that the benefit of corrected transitions is absorbed into the base policy through the critic and standard SAC updates.

5.5 Comparison with Hindsight Experience Replay

HER [Andrychowicz et al., 2017] is a replay augmentation technique originally designed for sparse, goal-conditioned settings: it relabels failed trajectories with achieved states as substitute goals, providing additional reward signal. HER is not ideally matched to the dense-reward Meta-World setting, but we include it as a test of whether generic relabeling-based replay augmentation addresses the same bottleneck problem as our method. The two approaches differ fundamentally: HER changes the reward label on existing transitions, while our method changes the action taken at stall states, producing transitions under a different policy.

Figure 5 and Table 5 show the push-v3 four-seed comparison.

HER achieves a mean final SR of 9.5%, below the 32.5% of plain SAC on the same seeds. Three failure patterns emerge across the four HER seeds: on seeds 0 and 2, the policy learns partially (peak 36% and 18%) but never converges; on seed 1, the policy fails completely; on seed 3, an alpha and Q-value explosion occurs (final alpha = 74.1, critic loss $> 3 \times 10^7$), turning the best baseline seed (70% final) into a complete failure. A likely contributing factor is a dense–binary reward mismatch: real push-v3 transitions carry dense rewards (episode totals 2000–3000), while HER

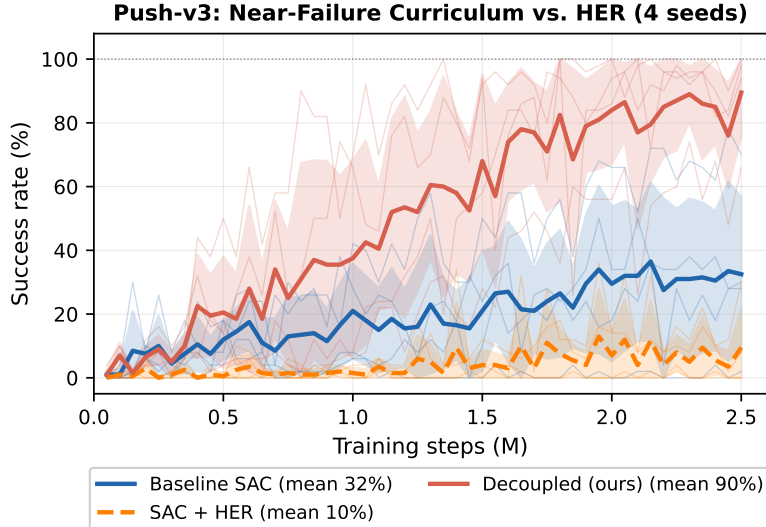


Figure 5: Push-v3: near-failure curriculum (ours) vs Hindsight Experience Replay vs baseline SAC (4 seeds each, 2.5M steps). HER, applied in a dense-reward setting it was not designed for, underperforms even the plain baseline. The 80 pp gap to our method (89.5% vs 9.5%) reflects different mechanisms rather than a matched comparison; see Section 6 for discussion of the dense–binary reward mismatch.

Table 5: Push-v3 comparison with Hindsight Experience Replay (4 seeds each, 2.5M steps). HER uses $her_ratio = 0.8$, future relabeling strategy, and binary relabeled reward. “Seeds conv.” counts seeds with peak SR $\geq 50\%$. Seed 3 of HER suffered an alpha/Q-value explosion (final alpha = 74, critic loss $> 3 \times 10^7$).

Condition	s0	s1	s2	s3	Mean final	Mean peak	Seeds conv.
Baseline SAC	30%	28%	2%	70%	32.5%	45.5%	2 / 4
SAC + HER	30%	0%	8%	0%	9.5%	16.0%	0 / 4
Decoupled (ours)	100%	98%	94%	66%	89.5%	93.0%	4 / 4

relabelled transitions carry binary rewards (0 or 1). The critic must learn consistent value estimates across these incompatible scales simultaneously, which may destabilize both the Q-function and the auto-alpha mechanism.

Our method avoids this mismatch entirely: corrected transitions carry the same dense MetaWorld reward as regular transitions. The correction improves the *action taken* at stall states, not the reward signal seen.

5.6 Ablation: Trigger Specificity

Figure 6 and Table 6 compare the heuristic trigger against a random trigger on push-v3. The random trigger fires at a fixed 5% rate per step with the same decoupled gradient isolation.

Table 6: Push-v3 trigger ablation: mean final success rate over 4 seeds. “Seeds conv.” counts seeds with peak success rate $\geq 50\%$. Random trigger fires at a fixed 5% rate per step with decoupled gradient isolation.

Condition	Mean final SR	Seeds conv.
Baseline SAC	32.5%	2 / 4
Decoupled + random trigger	36.0%	1 / 4
Decoupled + heuristic trigger	89.5%	4 / 4

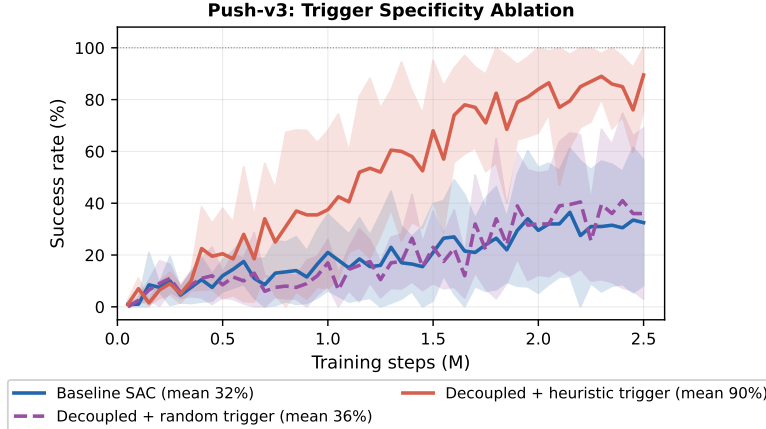


Figure 6: Push-v3 trigger specificity ablation (4 seeds each, 2.5M steps). The heuristic trigger substantially outperforms both the baseline and the random trigger. The random trigger, despite also using decoupled gradient isolation, performs no better than the baseline.

The random trigger (mean 36.0%) performs no better than the baseline (32.5%) and far below the heuristic (89.5%). Only one random-trigger seed converged past 50%. This confirms that the benefit is not achieved by applying the correction at some arbitrary fraction of steps: near-failure targeting is the essential ingredient. Random correction replaces transitions throughout training without distinguishing bottleneck states from states where the base policy is already performing well, providing no focused improvement to the critic’s Q-estimates at the states that matter.

5.7 Ablation: Latent vs. Action-Space Correction

We compare our latent-space correction (Δz added to the actor encoder output) against an action-space residual baseline (Δa added in pre-tanh action space, following Silver et al. 2018), with identical decoupled gradient isolation and heuristic trigger in both conditions. The action residual is regularized at $\lambda = 0.05$ with Δa clamped to $[-2, 2]$ to prevent tanh-saturation instability.

Table 7: Latent-space vs action-space residual correction: final success rate per seed (4 seeds each, 2.5M steps). All conditions use identical decoupled gradient isolation and heuristic trigger; only the correction space changes. “Conv.” counts seeds reaching $\geq 80\%$ final SR.

Method	s0	s1	s2	s3	Mean	Conv.
<i>Pick-place-v3</i>						
Baseline SAC	96%	82%	86%	0%	66.0%	3/4
Decoupled latent (ours)	100%	80%	98%	96%	93.5%	4/4
Decoupled action residual	0%	0%	92%	88%	45.0%	2/4
<i>Push-v3</i>						
Baseline SAC	30%	28%	2%	70%	32.5%	0/4
Decoupled latent (ours)	100%	98%	94%	66%	89.5%	3/4
Decoupled action residual	40%	4%	6%	54%	26.0%	0/4

Table 7 shows a consistent cross-task pattern: latent correction outperforms action residual by 48.5 points on pick-place and by 63.5 points on push. On push-v3, the action residual (26.0%) falls below the plain SAC baseline (32.5%) with no seed reaching 80% final SR, while latent correction reaches 89.5% mean with 3/4 seeds converging. On pick-place, action residual fails entirely on seeds 0 and 1 – seeds where baseline SAC alone succeeds at 96% and 82% – while latent correction succeeds on all four.

The failure mechanism on the difficult seeds is alpha collapse: the action residual fires on 60–87% of rollout steps during early training (when the policy stalls frequently), adding Δa directly to the sampled action and introducing additional anti-entropic pressure. This collapses α to the 0.05 floor

before Q-values can develop, blocking exploration before the bottleneck sub-task is discovered. Latent correction avoids this because Δz operates in unconstrained representation space before action sampling, leaving the SAC entropy dynamics unaffected.

5.8 Door-Open-v3: Both Methods Saturate

All four seeds of both baseline and decoupled reach 100% peak success on door-open-v3 within 500k steps. The method adds no benefit on tasks that baseline SAC already solves robustly, consistent with the curriculum framing: when the base policy is not stalling, the trigger rarely fires and the correction has little effect.

6 Discussion and Limitations

Why gradient isolation is primarily a training stability requirement. The most direct effect of the stop-gradient is that concurrent correction training becomes reliable: without it, joint gradient training fails to converge across seeds (0/4 on pick-place); with it, 4/4 seeds succeed on both tasks. A useful consequence is that the resulting base actor is also independently capable at deployment: base-off SR equals or exceeds triggered SR at every seed across both tasks (pick-place mean 95%, push mean 89.5%), confirming the benefit of corrected bottleneck transitions was absorbed into the base policy through the critic. The correction module may be retained at deployment if desired, but it is not required, and the base actor does not depend on it. The diagnostic always-on run (seed 42) shows the contrasting failure: joint gradient training produces a model where removing the correction causes a 60-point collapse (96% $\rightarrow$ 36%), because the encoder learned correction-compatible representations rather than independently good ones.

Why relabeling-based replay augmentation does not address the same bottleneck. HER was designed for sparse, goal-conditioned settings where the agent already visits useful states but receives no reward signal on them. It provides reward signal on states the policy already visits by substituting the achieved state as the goal, but it does not change the actions taken at states where the policy is stuck. Our method instead substitutes a sample from the corrected policy at stall states, changing what action is executed and empirically yielding stronger Bellman targets at bottleneck state-action pairs. The 80-percentage-point gap on push-v3 (89.5% vs 9.5%) likely reflects this difference, compounded by a dense-binary reward mismatch: push-v3 transitions carry dense rewards (episode totals 2000–3000), while HER relabeled transitions carry binary rewards, creating an inconsistent training signal for both the Q-function and the auto-alpha mechanism.

Assembly-v3: when trigger saturation prevents the curriculum. On assembly-v3, the heuristic trigger saturates: the stochastic policy oscillates during post-grasp transport, causing Phase B stall detection to fire at actual correction rates between 26% and 74% of steps across seeds. Rather than a targeted curriculum on genuine stall states, the correction becomes near-always-on, which collapses entropy before task discovery completes. The decoupled condition achieves below 5% final SR on all four seeds vs 98% for the best baseline seed. Figure 7 summarizes the cross-task pattern.

The pattern across tasks is coherent with the curriculum framing: the correction provides the most benefit on tasks of intermediate difficulty where the base policy can make partial progress but stalls on specific bottleneck states. On easy tasks the base policy never stalls long enough for the curriculum to matter. On hard tasks, the policy stalls so persistently that the trigger saturates and the correction becomes an obstacle rather than a tool.

Alpha stability and seed sensitivity. SAC’s auto-alpha mechanism is sensitive to the entropy of the initial policy distribution. Seeds where the initial weights produce concentrated actions early can collapse to low entropy, locking the policy before the bottleneck is discovered. The $\alpha_{\min} = 0.05$ floor mitigates the worst collapses but cannot direct exploration toward task-specific contact patterns. The correction bridges this gap for some seeds (pick-place seed 3, push seeds 1–2): the triggered correction generates grasping and contact transitions with better rewards and next states, consistent with stronger critic learning at bottleneck states and giving the base actor a stronger learning signal than it would receive from its own stalling transitions.

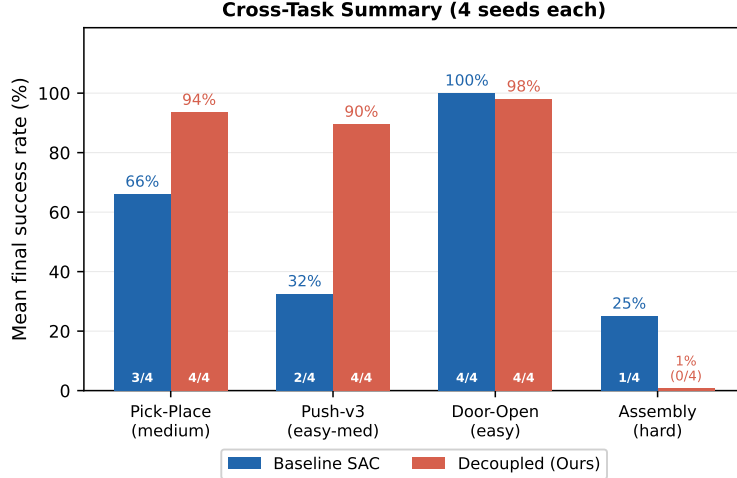


Figure 7: Cross-task summary (4 seeds each). The method provides the largest gains on push-v3 (medium difficulty) where the baseline SAC struggles to discover the contact-push sequence. On the easy door-open task both methods saturate. On the hard assembly task, trigger saturation causes near-always-on correction which collapses exploration before task discovery.

Why the correction must be concurrent, not sequential. We verify the concurrency requirement by training the same correction module sequentially: loading the converged baseline checkpoint, freezing the actor, and training only the correction for 1.5M additional steps. Table 13 (Appendix E) shows that sequential correction achieves 0% on pick-place seed 3 – the same as the unmodified baseline – while concurrent decoupled reaches 96%. On seeds where the baseline already converged (seeds 0–2), sequential correction provides at most 2 percentage points of improvement on seeds 0 and 2, while degrading seed 1 by 8 percentage points (82%→74%), far below concurrent decoupled’s gains, and with 60% more total compute (4M vs 2.5M steps). The correction’s benefit comes from providing stronger Bellman targets at bottleneck states *while the base actor is still learning from them*; once the base has converged, the Q-function has little room to improve and the correction module receives weak gradient signal.

Limitations. The heuristic trigger requires a task-specific subclass for each new task, encoding domain knowledge about which geometric signals indicate stalling. A learned trigger based on Q-advantage was implemented but failed due to label degeneracy: since the correction is trained to maximize Q , $\Delta Q > 0$ for almost all states at convergence, making binary trigger labels always positive and the predictor unable to learn selectivity.

Autograd gradients from the correction loss flow through the actor heads (Table 1), but `correction_optim` does not include actor-head parameters: only `actor_optim` in Step 2 updates them. Actor head weights are therefore not modified by Step 3; the gradient path exists in the autograd graph but does not affect parameters. Full autograd-graph isolation would require decoupled heads, but it has no practical consequence given the current optimizer scoping.

7 Conclusion

We proposed training a near-failure triggered latent correction module concurrently with an SAC base actor. When the trigger fires at a bottleneck state, the correction replaces the base policy’s action with a corrected action during rollout. The resulting transition enters the replay buffer and, if it reflects better task progress, provides a stronger Bellman target for the critic at that state. The base actor is updated by the standard SAC actor loss using this improved critic, benefiting from the corrected transitions indirectly. A stop-gradient on the actor encoder during the correction update ensures the base actor trains independently and does not develop a structural dependence on the correction.

Without gradient isolation, concurrent training causes co-adaptation: the actor encoder learns correction-compatible representations rather than independently good actions, degrading from 96% to

36% success when the correction is removed. With gradient isolation, the base actor at convergence performs identically with and without the correction active, confirming the benefit of corrected transitions was absorbed into the base policy.

Across pick-place-v3 and push-v3 in Meta-World 3.0, the method raises mean final success rate from 66.0% to 93.5% on pick-place and from 32.5% to 89.5% on push, primarily by rescuing seeds where baseline SAC fails or stalls severely, at the cost of minor regressions (2–4 pp) on two seeds that were already learning. A comparison against Hindsight Experience Replay on push-v3 (89.5% vs 9.5%) shows that targeted action correction at stall states addresses a different bottleneck than relabeling-based replay augmentation, likely compounded by a dense–binary reward mismatch that HER was not designed to handle.

The method does not help on assembly-v3, where trigger saturation causes the correction to fire on most steps, removing the selectivity that makes the approach effective. Designing a trigger that does not saturate on tasks requiring long exploration phases is the main open problem.

8 Contributions

The original project proposal listed a second group member. We decided to part ways, and all work in this report is solely mine.

Shou-Jen Chen completed all implementation, experiments, analysis, figures, and writing.

Detailed breakdown:

- **Literature review.** Surveyed residual policies, curriculum RL, training-only auxiliaries, and intervention policies to identify the gap this work fills.
- **Problem formulation.** Framed bottleneck sub-tasks as a replay-buffer quality problem. Discovered co-adaptation empirically (seed 42 base-off collapse) and identified it as the core challenge for concurrent correction training.
- **Algorithm design.** Designed the two-stage actor architecture with `z_override` injection and the three-step gradient isolation scheme (critic / base actor / frozen-encoder correction).
- **Agent implementation.** Implemented six agents in sequence: always-on correction, triggered correction, decoupled correction, action-space residual, sequential correction, and SAC+HER.
- **Trigger engineering.** Built heuristic stall detectors for four tasks: two-phase `PickPlaceTrigger` with drop detection; `PushTrigger` with a separate Phase B debug pass for contact transport; `DoorOpenTrigger` using `in_place_reward` as Phase B signal (Meta-World hardcodes `obj_to_target` to zero for door-open); `AssemblyTrigger`. Refactored all four into a factory pattern with a shared base class.
- **Learned trigger (not in paper).** Tried four versions of a Q-advantage-based learned trigger. All failed: at convergence $\Delta Q > 0$ for nearly every state, making binary trigger labels always positive.
- **Δz trigger (not in paper).** Tried firing on high $\|\Delta z\|$ magnitude. Failed: the threshold and signal co-evolve because trigger frequency controls module output magnitude.
- **TD3 extension (not in paper).** Implemented TD3 baseline, decoupled, and always-on. TD3 failed on pick-place (insufficient exploration without entropy) and door-open (local reward minimum at handle-reach). Excluded.
- **Debugging.** Fixed four issues: fixed-goal bug in early Meta-World setup; alpha collapse (log-space loss instability, added `alpha_min=0.05`); Q-value explosion on push seed 2 (gradient clipping was the wrong fix; alpha floor was the root solution); action residual explosion (`reg_coeff=0.05`, Δa clamping).
- **Infrastructure.** Set up Modal A100 remote training; managed volume downloads for pick-place seeds 2–3 and select ablation seeds.
- **Experiments.** Ran 4 seeds each of baseline and decoupled on four tasks, plus all ablations: random trigger, action-space residual, sequential correction, HER, base-off evaluation, co-adaptation diagnostic, and always-on multi-seed.

- **Figures and paper.** Generated all seven paper figures and wrote the full report.

References

- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- Philip J. Ball, Laura M. Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 1577–1594, 2023.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 41–48, 2009.
- Lakshita Dodeja, Karl Schmeckpeper, Shivam Vats, Thomas Weng, Mingxi Jia, George Konidaris, and Stefanie Tellex. Accelerating residual reinforcement learning with uncertainty estimation. *IEEE Robotics and Automation Letters*, 11:970–978, 2026. arXiv:2506.17564.
- Benjamin Eysenbach, Shixiang Gu, Julian Ibarz, and Sergey Levine. Leave no trace: Learning to reset for safe and autonomous reinforcement learning. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1587–1596, 2018.
- Alex Graves, Marc G. Bellemare, Jacob Menick, Rémi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1311–1320, 2017.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870, 2018.
- Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Proceedings of Robotics: Science and Systems (RSS)*, 2021.
- Xingyu Lin, Harjatin Baweja, George Kantor, and David Held. Adaptive auxiliary task weighting for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 4773–4784, 2019.
- Clare Lyle, Mark Rowland, Georg Ostrovski, and Will Dabney. On the effect of auxiliary tasks on representation dynamics. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 130 of *Proceedings of Machine Learning Research*, pages 1–9, 2021.
- Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.
- Remy Portelas, Cedric Colas, Katja Hofmann, and Pierre-Yves Oudeyer. Teacher algorithms for curriculum learning of deep RL in continuously parameterized environments. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 100 of *Proceedings of Machine Learning Research*, pages 835–853, 2020.

- Krishan Rana, Ming Xu, Brendan Tidd, Michael Milford, and Niko Suenderhauf. Residual skill policies: Learning an adaptable skill-based action space for reinforcement learning for robotics. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, volume 205 of *Proceedings of Machine Learning Research*, pages 2095–2104, 2023.
- Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.
- Brijen Thananjeyan, Ashwin Balakrishna, Suraj Nair, Michael Luo, Krishnan Srinivasan, Minh Hwang, Joseph E. Gonzalez, Julian Ibarz, Chelsea Finn, and Ken Goldberg. Recovery RL: Safe reinforcement learning with learned recovery zones. *IEEE Robotics and Automation Letters*, 6(3): 4915–4922, 2021.
- Ikechukwu Uchendu, Ted Xiao, Yao Lu, Banghua Zhu, Mengyuan Yan, Joséphine Simon, Matthew Bennis, Chuyuan Fu, Cong Ma, Jiantao Jiao, Sergey Levine, and Karol Hausman. Jump-start reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 34556–34583, 2023.
- Nolan C. Wagener, Byron Boots, and Ching-An Cheng. Safe reinforcement learning using advantage-based intervention. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 10630–10640, 2021.
- Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 100 of *Proceedings of Machine Learning Research*, 2020.
- Xiu Yuan, Tongzhou Mu, Stone Tao, Yunhao Fang, Mengke Zhang, and Hao Su. Policy decorator: Model-agnostic online refinement for large policy model. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.13630.

A Hyperparameters and Architecture

Table 8: SAC hyperparameters. All conditions (baseline, always-on, decoupled) use identical SAC settings.

Parameter	Value	Notes
Hidden dim (actor, critic)	256	
Hidden layers	2	
Actor learning rate	3×10^{-4}	
Critic learning rate	3×10^{-4}	
Discount γ	0.99	
Soft-update τ	0.005	
Batch size	128	
Replay capacity	1,000,000	
Warmup steps	5,000	Uniform random actions
Updates per step	1	
Target entropy $\mathcal{H}_{\text{target}}$	$-4 (= - \mathcal{A})$	
$\alpha_{\min}$	0.05	Log-space clamp after each update
α_{lr}	1×10^{-4}	
Total steps	2,500,000	1,000,000 for door-open
Eval frequency	50,000 steps	
Eval protocol	All 50 MT1 variants, fixed order	

B Per-Seed Result Tables

All numbers are verified from training logs. Peak SR is the maximum success rate observed at any eval checkpoint. Final SR is the success rate at the last recorded eval step (2,500,000 for most runs). Steps to 80% is the first step at which success rate reached 80%; “—” means never reached.

Table 9: Correction module and trigger hyperparameters.

Parameter	Value	Notes
Correction hidden dim	128	2 ReLU layers
Correction output scale	0.1	Applied before adding Δz
Correction output init gain	0.01	Orthogonal init; near-identity at start
Correction learning rate	3×10^{-4}	
Regularisation λ	10^{-3}	$\lambda \ \Delta z\ ^2$ added to correction loss
Trigger window W	20 steps	Sliding window for stall detection
Trigger min-progress $\delta_{\min}$	0.01 m	Range threshold
Trigger cooldown	15 steps	Active steps after each fire event
Trigger success threshold	0.05 m	Suppresses trigger near goal
Pregrasp close (pick-place)	0.04 m	Suppresses Phase A near contact
Pregrasp close (push)	0.06 m	Phase A/B boundary

Table 10: Pick-place-v3 per-seed results (2.5M steps each). Baseline s0 folder: sac_pp_baseline_s0_pilot; s2, s3: Modal A100. Decoupled s0, s1: local MPS; s2, s3: Modal A100.

Condition	Seed	Peak SR	Peak step	Final SR	Steps to 80%
Baseline	s0	100%	1,700k	96%	1,550k
	s1	100%	1,750k	82%	1,100k
	s2	100%	2,250k	86%	1,200k
	s3	0%	—	0%	—
Decoupled	s0	100%	1,800k	100%	1,400k
	s1	92%	2,350k	80%	2,200k
	s2	100%	2,100k	98%	1,750k
	s3	100%	2,300k	96%	1,750k

C Gradient Isolation: Implementation Details

The three-step update in the decoupled agent (`src/agents/sac_decoupled.py`) is implemented as follows. Step 1 (critic) is identical to standard SAC. Step 2 (base actor) is also standard SAC, with no reference to the correction module. Step 3 (correction) uses `torch.no_grad()` around the encoder call:

```
# Step 3: correction update (encoder is frozen)
with torch.no_grad():
    z_base = self.actor.encoder(obs)          # detached tensor
    delta_z = self.correction(obs, z_base)
    a_corr, log_pi_corr, _ = self.actor.forward(
        obs, z_override=z_base + delta_z)
    loss_corr = (alpha * log_pi_corr
        - min(Q1(obs, a_corr), Q2(obs, a_corr))
        + lam * delta_z.pow(2).mean())
    self.correction_optim.zero_grad()
    loss_corr.backward()
    self.correction_optim.step()
```

The `no_grad()` wrapper creates `z_base` as a tensor with `grad_fn = None`, so `loss_corr.backward()` propagates no gradients to `self.actor.encoder`. The actor heads receive gradients from Step 3 through `z_override` and the heads' own forward pass, but the encoder does not.

Table 11: Push-v3 per-seed results (2.5M steps each, all local MPS). Decoupled uses the two-phase trigger (v2). Baseline s0 peak is exactly 50%; no eval step exceeded 50%.

Condition	Seed	Peak SR	Peak step	Final SR
Baseline	s0	50%	1,950k	30%
	s1	36%	2,150k	28%
	s2	18%	500k	2%
	s3	78%	2,450k	70%
Decoupled	s0	100%	1,350k	100%
	s1	100%	1,800k	98%
	s2	98%	2,250k	94%
	s3	74%	2,300k	66%

Table 12: Assembly-v3 per-seed results (2.5M steps each, all local MPS). Decoupled s3 final SR is 4%, not 0%; this reflects a single noisy eval episode succeeding at the last checkpoint, not sustained convergence.

Condition	Seed	Peak SR	Final SR	Notes
Baseline	s0	0%	0%	Never converged
	s1	98%	98%	Only seed that learned
	s2	0%	0%	Never converged
	s3	14%	2%	Late transient, regressed
Decoupled	s0	4%	0%	Transient peak only
	s1	0%	0%	Never converged
	s2	28%	0%	Transient peak, regressed
	s3	4%	4%	Single noisy eval at 2,500k

D Trigger Pseudocode (Pick-Place-v3)

The following pseudocode describes the `PickPlaceTrigger.update(obs, info)` method. Other tasks use the same structure with different progress signals.

```

update(obs, info):
    grasp = info["grasp_success"]
    obj_dist = info["obj_to_target"]
    hand_to_obj = norm(obs[0:3] - obs[4:7])

    if obj_dist < 0.05:          # success suppression
        reset_state(); return False

    if prev_grasp and not grasp and obj_dist > 0.05:  # drop
        postgrasp_hist.clear(); fire(cooldown=15)

    if grasp != prev_grasp:      # phase transition
        pregrasp_hist.clear(); postgrasp_hist.clear()

    if cooldown_left > 0:
        cooldown_left -= 1; return True

    if not grasp and hand_to_obj > 0.04:  # Phase A
        pregrasp_hist.append(hand_to_obj)
        if full(pregrasp_hist) and range(pregrasp_hist) < 0.01:
            fire(cooldown=15); return True

    if grasp:                      # Phase B
        postgrasp_hist.append(obj_dist)
        if full(postgrasp_hist) and range(postgrasp_hist) < 0.01:
            fire(cooldown=15); return True

```

```
return False
```

E Sequential vs Concurrent Correction

We test whether concurrent training is necessary by comparing against a sequential protocol: the converged baseline checkpoint is loaded, the actor is frozen, and only the correction module trains for 1.5M additional steps using the same trigger and gradient isolation as concurrent decoupled. The frozen base SR at the start of each sequential run was verified by evaluating the loaded checkpoint before any correction training.

Table 13: Sequential vs concurrent correction on pick-place-v3 (4 seeds each). Sequential: load converged baseline, freeze actor, train correction for 1.5M additional steps (4M total compute). Concurrent decoupled: train base and correction together for 2.5M steps. “Frozen base SR” is the baseline checkpoint SR at the start of sequential correction training, verified from the logged step-0 evaluation.

Method	s0	s1	s2	s3	Mean	Seeds conv.
Baseline SAC (frozen base)	96%	82%	86%	0%	66.0%	3/4
Sequential correction	98%	74%	88%	0%	65.0%	2/4
Concurrent decoupled (ours)	100%	80%	98%	96%	93.5%	4/4

Sequential correction achieves 0% on seed 3, where the baseline itself also fails and concurrent decoupled reaches 96%. On seeds where the baseline already converged (seeds 0–2), sequential correction adds at most 2 percentage points over the frozen base and slightly degrades seed 1 (82% $\rightarrow$ 74%). The correction module’s Δz norm reaches only 0.8–1.8 in sequential training (vs 8–12 in concurrent decoupled), because the Q-function has little room to improve on an already-converged policy. Sequential correction also uses 60% more total compute (4M steps vs 2.5M).